AEGIS: A Constraint-Based Architecture for Safe LLM-Assisted Natural Language Analytics

Md. Riaz (ORCID: 0009-0003-8850-4122)[1*]

[1*]Department of Computer Science and Engineering, Pundra University of Science and Technology, Bogura, Bangladesh.

Corresponding author(s). E-mail(s): mdriaz.wd@gmail.com;

Abstract

Natural-language analytics can make relational business data usable for non-technical users, but direct text-to-SQL generation is hard to control in operational reporting: a syntactically valid query can still violate business definitions, expose unauthorized data, or produce an unsafe report. This paper presents AEGIS, a constraint-based architecture for safe LLM-assisted natural-language analytics. AEGIS restricts the language model to typed intent extraction over an approved semantic layer; rule-based components then validate that intent, plan the query, compile SQL from templates, check it, select a visualization, and persist a reusable widget. It is evaluated on a nopCommerce deployment with two static, repeatable materials: a 500-question benchmark (425 supported and 75 realistic boundary requests) and nopCommerce's own twenty standard admin reports, whose shipped implementations serve as an independent reference. AEGIS parses 499/500 prompts, answers 423/425 supported requests and executes 422 (99.3%), and declines 72/75 boundary requests (96.0%). Against the platform's own report logic, fifteen of twenty result sets match exactly; the other five agree on every value and differ only in row count and label column. The same model writing SQL directly, with no semantic layer, executes 365/425 supported requests (85.9%) but answers 25 of the 75 unanswerable questions anyway (33.3%) and emits two queries with forbidden constructs. Intent extraction dominates latency at a 9.0 s median, while every rule-based stage after it completes in 3.6 ms. AEGIS is therefore not an unrestricted natural-language-to-SQL engine but a bounded architecture for safe, reusable analytics over an explicitly approved semantic layer.

Keywords: natural language interfaces, text-to-SQL, semantic layer, dashboard generation, LLM safety, constrained query generation

1 Introduction

Organizations store much of their operational knowledge in relational databases: sales transactions, customer records, inventory, payments, shipments, refunds, and similar business data. Although this data is valuable for decision-making, access to it is uneven. Technical users can write SQL queries directly, while non-technical users often depend on developers, analysts, or administrators to prepare reports for them. This creates delay and also discourages repeated exploratory questions such as changing a date range, comparing another product group, or viewing the same metric by a different dimension.

Natural language interfaces to databases (NLIDBs) address this problem by allowing users to ask questions in ordinary language. Recent neural text-to-SQL systems and large language models have improved substantially on benchmark datasets, but practical reporting systems require more than producing a plausible SQL query. In an operational dashboard, the answer must respect business definitions, database permissions, safety constraints, and presentation requirements. A generated query that is syntactically valid can still be unsafe, semantically wrong, or unsuitable for reuse.

This paper presents **AEGIS** (Analytics Engine with Guaranteed Injection Safety), a constraint-based architecture for safer natural-language analytics. AEGIS does not ask the language model to generate SQL. Instead, the model extracts a structured analytical intent from the user's request. The rest of the pipeline maps that intent to an approved semantic layer, compiles SQL from rule-based templates, validates the query, executes it, selects a visualization, and stores the result as a reusable dashboard widget.

The central idea is to separate language understanding from query execution. The language model is useful for interpreting user wording, but it is not trusted with database structure, SQL syntax, access control, or business definitions. Those responsibilities remain in explicit system components controlled by the application owner. If a request cannot be expressed using the approved semantic layer, AEGIS returns a clarification or rejection rather than substituting the nearest available metric and silently producing a wrong answer.

This work contributes:

1. A reporting-oriented architecture that converts natural-language requests into persistent dashboard widgets rather than one-off SQL query results.
2. An approved semantic-layer contract for declaring metrics, dimensions, joins, filters, time rules, permissions, and visualization defaults.
3. A rule-based SQL compilation pipeline that generates read-only queries from approved analytical templates instead of model-authored SQL.
4. A matching and structured-intent validation mechanism that separates answerable requests from unsupported or ambiguous requests before query compilation.
5. A repeatable nopCommerce evaluation protocol that pairs breadth over 500 natural-language requests with accuracy against the platform's own admin-report implementations as an independent reference, and reports rejection of out-of-boundary requests alongside the answer rate that bounds it.

6. Experimental evidence that a bounded semantic-layer system can provide high supported-request execution while preserving explicit rejection behavior for realistic out-of-boundary questions.

The paper is organized as follows. Section 2 reviews related work in natural-language interfaces, text-to-SQL, natural-language visualization, dashboard generation, and semantic layers. Section 3 presents the analytical task classification that motivates the template library. Section 4 presents the AEGIS architecture. Section 5 describes the prototype implementation. Section 6 reports the evaluation. Sections 7-9 discuss implications, limitations, and conclusions.

2 Related Work

2.1 Natural Language Interfaces to Databases

Natural language database interfaces have been studied for over four decades. Early systems such as LUNAR [1] and TEAM [2] used hand-crafted grammars and domain-specific ontologies to parse queries. These systems were brittle under vocabulary variation but established the core insight that query understanding requires a bridge between natural language and schema semantics. One way to build that bridge is an explicit business ontology or metadata layer. SODA [3] generates SQL for business users by matching their words against database metadata and an ontology, and ATHENA [4] answers a question by first mapping it to an ontology and then translating that mapping to SQL. AEGIS keeps this idea of a curated business layer between the user and the tables, but adds explicit refusal and reusable widgets on top of it.

NaLIR [5] is an important modern NLIDB because it treats ambiguity as a real problem to solve rather than an error. By showing users different possible interpretations of their question, NaLIR improves accuracy but requires the user to actively participate. AEGIS uses a similar approach - asking for clarification when the meaning is unclear - but extends it into a full widget lifecycle that NaLIR does not cover. Survey work [6, 7] confirms that ambiguity, portability, schema complexity, and controlled access remain ongoing challenges across NLIDB generations and are not solved by bigger models alone.

2.2 Neural Text-to-SQL and Benchmark Progress

Seq2SQL and WikiSQL moved the field toward neural approaches [8], which showed that aligned training data could teach models to produce SQL. Spider [9] advanced the challenge significantly by introducing cross-domain schemas and complex multi-table queries, becoming the standard benchmark. SParC and CoSQL [10, 11] extended the evaluation to conversational and contextual settings. BIRD [12] brought benchmark queries closer to production conditions by emphasizing large databases, value matching, and query efficiency.

Schema-aware encoding, introduced in RAT-SQL [13], showed that explicitly modeling schema relationships improves accuracy on new databases. Constrained decoding approaches such as PICARD [14] showed that rejecting invalid SQL tokens during generation improves results. Large language models then shifted the field from training a

specialized parser to prompting a general model. DIN-SQL [15] breaks the task into schema linking, classification, generation, and self-correction; DAIL-SQL [16] studies which worked examples and prompt formats help the model most; and multi-agent methods such as MAC-SQL [17] and CHESS [18] divide generation, schema selection, and checking among cooperating agents. Benchmarks grew harder alongside them, with Spider 2.0 [19] replacing single questions with multi-step enterprise reporting workflows over large schemas. More recent systems like G-SQL [20] and TriSQL [21] add rule guidance and multi-stage checking. These systems improve text-to-SQL accuracy, but they still focus on SQL generation. They do not address safe data access, permission control, widget storage, or chart selection, which are the concerns AEGIS targets.

A smaller line of work asks whether a question should be answered at all. TriageSQL [22] treats this as a classification problem, sorting a question into answerable or one of several kinds of unanswerable before any SQL is written, and EHRSQL [23] builds a hospital-records benchmark that deliberately mixes answerable and unanswerable questions and rewards a model that declines when it is not confident. AEGIS shares this concern but makes the decision a built-in outcome of the pipeline: an unsupported request is refused or sent back for clarification rather than answered with a guessed query.

2.3 Natural Language for Visualization

A parallel research stream focuses on NL-driven chart generation rather than SQL generation. nl4dv [24] maps natural language queries to analytic tasks and visual encodings. nvBench [25] introduced a cross-domain benchmark for NL-to-visualization. Eviza [26] enabled conversational interaction with existing visualizations. DataTone [27] managed ambiguity in NL visualization interfaces through mixed-initiative interaction, surfacing alternative chart interpretations to users - a concept AEGIS adopts in its clarification model.

2.4 Dashboard Generation

Recent work also treats dashboard generation as an automated design problem. DashBot [28] proposed using deep reinforcement learning to compose dashboards from a set of data insights. MultiVision [29] used bidirectional LSTM models to score individual charts and combine them into multi-view dashboards. DataShot [30] and Calliope [31] used statistical fact extraction followed by template-based layout to generate narrative data documents.

2.5 Semantic Layers and Controlled Analytics

A semantic layer is a business-logic abstraction that maps business concepts to the actual database tables and columns. Commercial tools like dbt Metrics, Looker LookML, and Apache Superset implement semantic layers in different ways. Newer metric layers such as Cube, dbt's MetricFlow, and Snowflake's Cortex Analyst push the same idea further by exposing a fixed set of approved metrics to downstream tools, so that every query uses one agreed definition instead of ad hoc SQL. Lehmann et al.

[32] stress the importance of controlled data access in practical NL database interfaces. The risk of letting a model write the SQL is not only that the answer may be wrong: Pedro et al. [33] show that applications which build SQL from model output can be driven to run attacker-chosen queries through the prompt itself, a prompt-to-SQL injection, and AEGIS closes that path by never turning model output into SQL. Structured output enforcement for LLMs [34] has been shown to improve the reliability of typed object generation, which AEGIS uses for intent extraction. The reviewed work does not use a semantic layer as the main safety mechanism for an LLM-assisted reporting system.

2.6 Comparative Summary

Table 1 positions AEGIS against the systems reviewed above along the capabilities this paper is concerned with: natural-language parsing, an explicit semantic layer, safe SQL construction, visualization, widget persistence, structured intent validation, and the kind of evaluation each system reports. Read down the AEGIS row against the others and the table's point is clear: prior systems each cover part of this pipeline — parsing and SQL generation, chart rendering, controlled access, or deciding whether a question can be answered — while AEGIS combines a semantic layer, rule-based SQL construction, and structured intent validation in one reporting pipeline. It is also the only row whose evaluation measures refusal of out-of-scope questions and checks fidelity against a platform's own report code, rather than reporting a benchmark score alone.

3 Analytical Task Classification

3.1 Classification Construction

The eleven analytics primitives in Table 2 were identified through a review of representative e-commerce and administrative reporting requests conducted during AEGIS design. This classification is a design choice: it defines the finite request shapes the compiler is allowed to render, rather than claiming that all possible user questions fit these shapes.

The evaluation does not draw its evidence from a general-purpose cross-domain benchmark, because such benchmarks measure generalisation to unseen schemas rather than the property this architecture claims. It uses a static nopCommerce dataset with 500 natural-language questions: 425 supported questions that should be answerable by the implemented semantic layer and 75 realistic e-commerce boundary questions that should be rejected or clarified.

3.2 Position of the Evaluation Dataset

The benchmark is finite by design. It measures whether the implemented nopCommerce semantic layer covers useful combinations of approved metrics, dimensions, time rules, predicates, and analytical patterns. It does not measure open-ended text-to-SQL capability. This matches the AEGIS claim: useful natural-language analytics should be broad within the declared semantic layer and explicit outside it.

Table 1 Comparative positioning of AEGIS against related systems.

System	NL	Sem.	Safe SQL	Viz.	Widget	Intent Validation	Evaluation
Spider / BIRD (Yu '18; Li '23)	Yes	-	-	-	-	-	Benchmark
Seq2SQL (Zhong '18)	Yes	-	-	-	-	-	Benchmark
RAT-SQL (Wang '20)	Yes	-	-	-	-	-	Benchmark
PICARD (Scholak '21)	Yes	-	Partial	-	-	-	Benchmark
NaLIR (Li '14)	Yes	-	-	-	-	-	Benchmark
nl4dv (Narechania '21)	Yes	-	-	Yes	-	-	In-memory
DashBot (Deng '23)	-	-	-	Yes	Partial	-	Synthetic
Lehmann et al. [32]	-	Yes	-	-	-	-	Position
LLM prompting (DIN-SQL, DAIL-SQL)	Yes	-	-	-	-	-	Benchmark
Abstention (TriageSQL, EHRSQL)	-	-	-	-	-	Partial	Benchmark
Metric layers (Cube, MetricFlow)	-	Yes	Partial	-	-	-	Industry tool
AEGIS (this work)	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**	**nopCommerce evaluation**

The 500-question dataset, the extracted semantics of nopCommerce's twenty standard admin reports, and their reference queries are static files so that the claims can be inspected and rerun.

4 The AEGIS System

AEGIS divides responsibility between the language model and the rest of the system. The language model interprets the user's wording; all database-facing behavior is controlled by rule-based components.

4.1 Design Principles

AEGIS follows five design principles.

1. **Separate understanding from execution.** The LLM is used for intent extraction, not SQL generation.
2. **Represent business meaning explicitly.** Metrics, dimensions, filters, joins, and time rules are declared in a semantic layer.
3. **Constrain query construction.** SQL is produced only by rule-based templates over approved semantic-layer objects.

Table 2 Analytical task classification used by the AEGIS compiler.

Pattern	Purpose	Example
KPI / Aggregate	Single-number business summary	Total revenue this month
Ranking	Top or bottom entities by metric	Top products by revenue
Exception / Filter	Rows that meet an operational condition	Low-stock products
Trend Analysis	Metric over time	Monthly sales trend
Comparison	Metric compared across groups or periods	Revenue by order status
Summary / Group	Grouped business overview	Orders by payment status
Cohort	Population split by lifecycle stage	New versus returning customers
Funnel	Stepwise business process	Checkout progression
Correlate	Relationship between two measures	Discount versus order value
Segment	Breakdown by business dimension	Revenue by country
Tabular	Detailed record listing	Latest orders

4. **Treat non-answer as a valid outcome.** Unsupported or ambiguous requests should be rejected or clarified rather than forced into the nearest available query.
5. **Produce reusable analytical outputs.** The output is not only a query result, but a dashboard widget with visualization and refresh behavior.

4.2 System Overview

Figure 1 shows the full path from request to widget. The pipeline begins with a request such as "Which products brought in the most revenue this month?" The LLM converts the sentence into a structured intent: a ranking request, using the revenue metric, grouped by product, filtered to the current month. AEGIS then checks whether each requested concept is present in the semantic layer. If the concepts are available, the analysis planner builds a standard plan. If a concept is missing, such as a marketing campaign dimension that has not been modeled, the request is rejected or clarified.

The compiler then converts the plan into SQL using approved templates. The LLM does not choose table names, join clauses, predicates, or SQL syntax. After compilation, a safety scanner verifies that the query is read-only and contains no forbidden constructs. The query is then executed, a visualization is selected, and the result is stored as a reusable widget.

The pipeline is:

User Request -> LLM Intent Parser -> Structured Intent Validator -> Semantic Mapper -> Analysis Planner -> Safe Query Compiler -> Permission Rewriter -> Query Executor -> Visualization Selector -> Widget Engine -> Dashboard

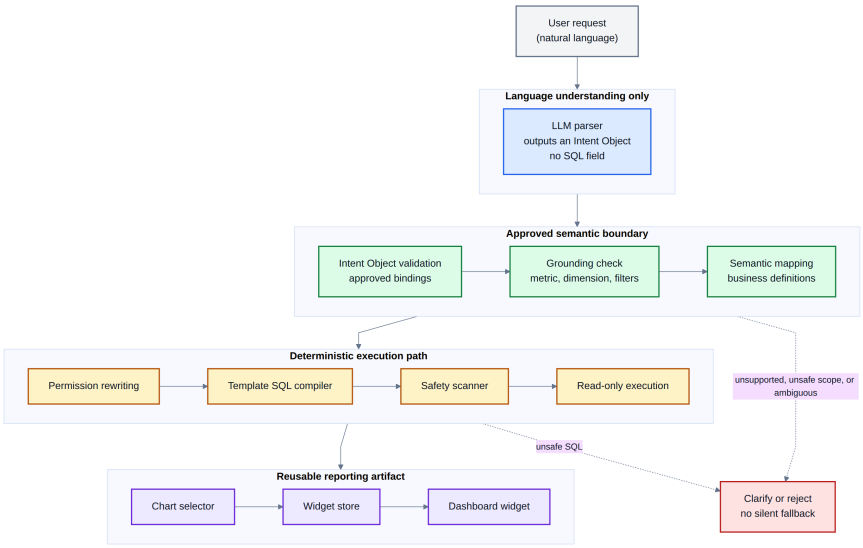


Fig. 1 AEGIS architecture pipeline from natural-language request to reusable dashboard widget.

This staged design separates AEGIS from direct LLM-to-SQL systems. A direct system asks the model to produce executable SQL; AEGIS asks the model only to describe the user's analytical intent.

4.3 Semantic Layer

The semantic layer is the main control point in AEGIS, and Fig. 2 shows how its approved concepts are composed before any SQL is produced. It defines the business concepts that the system is allowed to answer. A metric specifies a measurable quantity such as revenue, order count, refund amount, or customer count. A dimension specifies how a result can be grouped or filtered, such as product, category, country, order status, payment status, or month. The semantic layer also records required joins, mandatory predicates, access rules, and default visualization choices.

This layer separates business language from the physical database schema. A user may ask for "sales", "amount spent", or "revenue", but the system maps those expressions to one approved metric definition. The same mechanism prevents unsupported concepts from being silently substituted. If the semantic layer does not define campaign attribution, review sentiment, or forecasted demand, AEGIS should not invent a query for those concepts.

4.4 Intent Parsing with Vocabulary Injection

AEGIS builds the LLM prompt from the semantic layer at runtime. The prompt lists the approved metrics and dimensions with short descriptions, and instructs the model to return a typed JSON intent object. This is called vocabulary injection. It allows the model to map flexible user wording onto approved identifiers without maintaining a separate synonym dictionary.

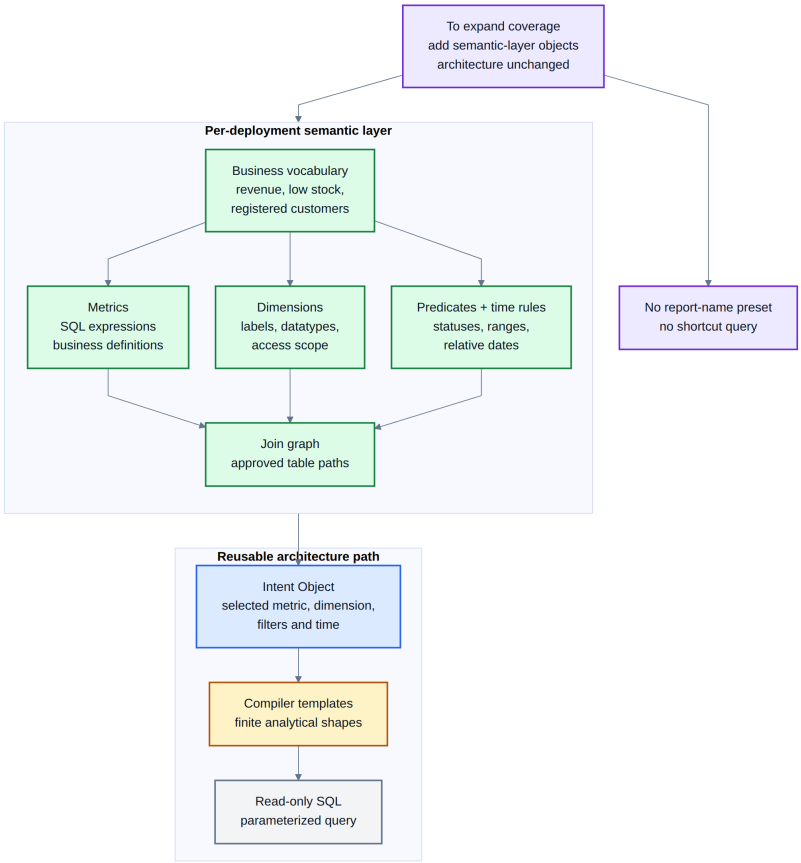


Fig. 2 Semantic-layer modularity: Approved business concepts are composed before SQL is compiled.

The parser output contains fields such as the analytical pattern, metric, dimension, time phrase, filters, sort order, and limit. For example, a request for "top products by revenue this month" should produce a ranking intent with `revenue` as the metric, `product_name` as the dimension, descending sort order, and a current-month time rule. The parser does not produce SQL.

4.5 Matching and Structured Intent Validation

The matching stage verifies that each parsed term corresponds to an approved semantic object. It returns one of three outcomes for each important slot: resolved, ambiguous, or unsupported. This prevents silent fallback from one business concept to another.

AEGIS validates the structured intent produced by the LLM against approved metrics, dimensions, filters, analytical patterns, and join rules. The original request text is not treated as a broad vocabulary checklist, because vague and multilingual wording is exactly what the LLM is meant to standardize. Raw text is retained only

for narrow non-executable cues: destructive write requests, direct credential or secret requests, and explicit prediction, causal explanation, or sentiment-analysis requests outside the SQL-only prototype scope. If the LLM misinterprets vague unsupported language as a supported intent, that remains an intent-extraction limitation reduced by confidence, clarification, and visible debug traces; the rule-based compiler can only guarantee that any executed SQL comes from approved semantic definitions.

4.6 Time Grammar and Analysis Planning

Time expressions are standardized by a dedicated time grammar. Phrases such as ”today”, ”this month”, ”last 30 days”, and ”monthly” are converted into explicit time rules. Unsupported time phrases are reported rather than ignored. This avoids a common reporting error in which a time filter is dropped and the query silently runs over all available data.

After matching and time standardization, AEGIS builds an analysis plan. The plan records the approved metric, dimension, filters, time rule, visualization pattern, sort order, and limit. This plan is the contract between natural-language interpretation and rule-based query compilation.

4.7 Safe Query Compiler

The compiler converts an analysis plan into SQL by expanding approved templates. Figure 3 shows the two layers of SQL safety this produces: structural prevention during compilation, and validation of the compiled query afterwards. It selects the required tables, resolves join paths from the semantic-layer graph, applies mandatory predicates such as soft-delete filters, binds user values as parameters, and adds grouping, ordering, and limits according to the analytical pattern.

The compiler also handles reporting-specific correctness rules. For example, an order-level metric such as total revenue cannot be grouped directly by product category without double-counting orders that contain multiple line items. In such cases, the semantic layer can define an item-grain equivalent metric, and the planner can use that safer definition for product-level breakdowns.

The compiled SQL is then checked by a post-compilation safety scanner. Queries containing write operations, system-table access, or other forbidden constructs are rejected. This safety property holds by design, under the assumptions stated in Section 8, because SQL structure comes from templates and semantic-layer definitions rather than from untrusted natural-language text.

4.8 Visualization and Widget Generation

Once a query executes, AEGIS selects a visualization based on the analytical pattern and result shape. Figure 4 shows the resulting widget lifecycle. Scalar results become KPI cards, ranked lists become bar charts or tables, trends become line charts, and tabular results remain tables. The widget engine stores the generated output so that users can refresh and reuse the report rather than asking the same question repeatedly.

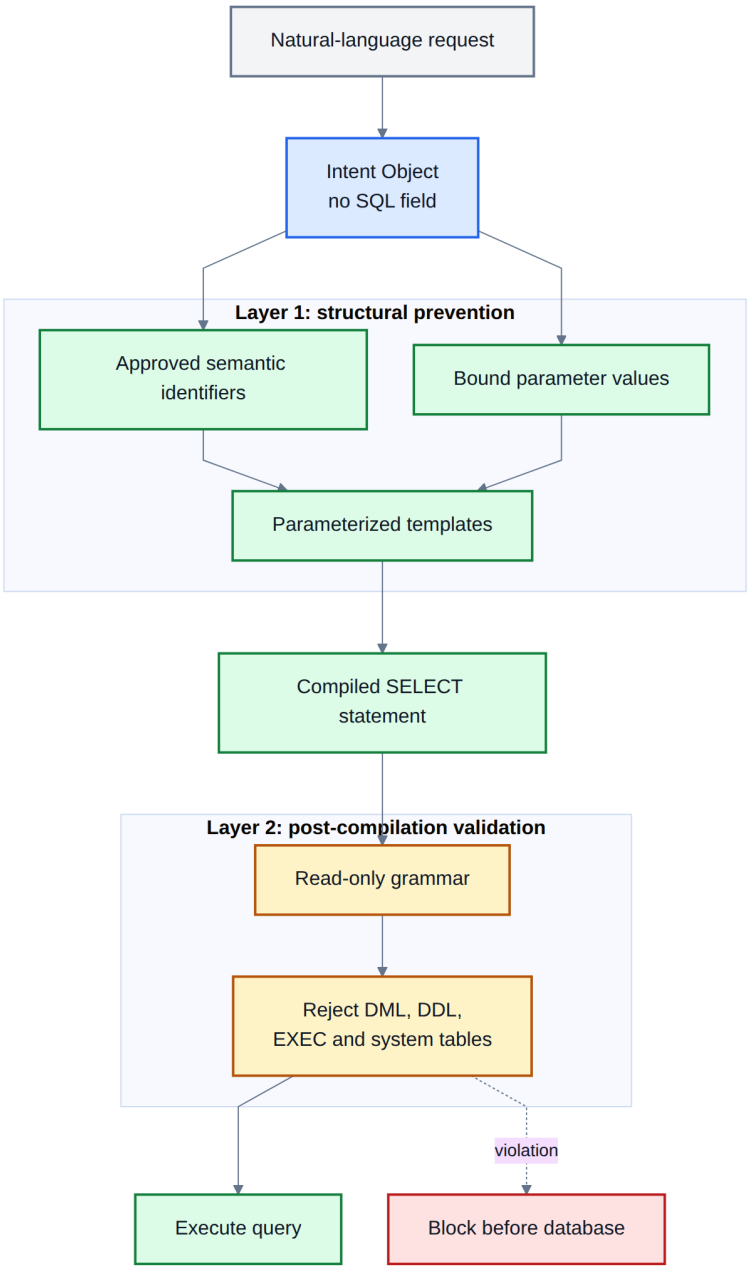


Fig. 3 Two-layer SQL safety model combining structural prevention with post-compilation validation.

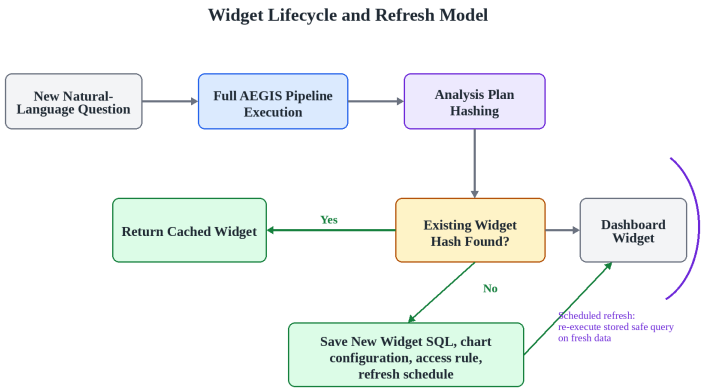


Fig. 4 Widget lifecycle for reusable natural-language analytics.

4.9 Terminal Outcomes

AEGIS has three terminal outcomes:

- **ANSWER:** the request is supported, SQL is compiled, and a widget is produced.
- **CLARIFY:** the request is potentially answerable but ambiguous, so the system asks a specific follow-up question.
- **REJECT:** the request depends on concepts or operations outside the approved semantic layer.

This explicit outcome model keeps unsupported requests visible rather than hiding them as crashes or missing features.

5 Implementation

AEGIS is implemented as a web application with a plain HTML/JavaScript frontend (jQuery, Chart.js) and a Python (FastAPI) backend targeting a nopCommerce-derived MySQL schema of 126 tables and 107 foreign-key constraints. The reference queries in Section 6.4 are read from nopCommerce source at commit `64bdf2ff08c8b39e65717bcf974fb43dc2ef68f2` (version 5.00.0); all twenty executed against this schema without a missing table or column, which establishes that the schema and the shipped report logic agree on the entities those reports read.

- **LLM Integration:** An LLM API exposed through an OpenAI-compatible `/v1/chat/completions` interface, reached through the `CUSTOM` provider profile in `aegis/server/ai_config.py` whenever `LLM_BASE_URL` is set, with `LLM_MODEL` naming the model to request. AEGIS uses structured JSON output enforcement and a system prompt constructed by injecting approved metric and dimension IDs. The reported evaluation used this OpenAI-compatible LLM API; the SQL compiler and safety layer are independent of the provider because they consume only the typed intent object.

- **Rate Limiting:** Provider-agnostic configuration module (`ai_config.py`) with sliding-window rate limiter and concurrency-safe `asyncio.Lock`.
- **Semantic Layer:** Python configuration modules containing the approved nopCommerce metrics, dimensions, predicates, and join paths used by the static evaluation dataset. The implementation deliberately keeps synonyms out of a separate hand-maintained dictionary; wording coverage is handled through dynamic vocabulary injection over semantic-layer descriptions.
- **SQL Compiler:** Parameterized MySQL templates. BFS join path resolution across 14 tables (12 aliases). Post-compilation `_validate_sql_safety()` checks 16 forbidden patterns.
- **Visualization Selector:** Rule-based Python dictionaries. Additional rules after data: bar charts with >20 categories become tables, pie charts with >8 slices become bar charts.
- **Widget Engine:** SHA-256 plan hash deduplication. JSON file storage in prototype (designed for relational database in production).
- **Structured Intent Validator:** Pre-compilation gate rejects unsupported or ambiguous structured metric/dimension/filter bindings, and declines narrow raw-text safety/scope cues such as writes, direct secrets, and explicit non-SQL analytics modes.
- **Permission Enforcement:** A Permission Rewriter appends role-based WHERE predicates after compilation, so no natural-language content can influence them. Five roles are defined (`public`, `dept_chair`, `regional_manager`, `read_only`, `analyst`), but this stage is a demonstration of the pattern rather than enforced access control: the predicates are illustrative, role values are inserted rather than bound, and the stage is outside the evaluated path of Section 6. Section 8 records this.

6 Evaluation

This section evaluates AEGIS on the nopCommerce e-commerce deployment using static, repeatable datasets. The evaluation separates three concerns that natural-language analytics papers often mix: broad request coverage over the implemented semantic layer, accuracy to first-party reporting semantics where source-derived references exist, and correct refusal of plausible requests outside the declared semantic boundary.

Metrics. Each reported figure is a ratio over a fixed denominator, defined here so the tables can be read without ambiguity. Parser success is the fraction of the 500 prompts for which the model returns a well-formed typed intent object that parses into the expected schema, independent of whether the extracted intent is correct. Supported intent exact match is the fraction of the 425 supported questions whose parsed intent agrees with the committed annotation on every slot: analytical class, metric, dimension, time phrase, filters, sort order, and limit. Supported answer rate is the fraction of supported questions that reach the ANSWER outcome rather than being rejected or clarified. Supported execution validity is the fraction of supported questions whose compiled SQL executes against the database without error. Boundary

Table 3 Evaluation dataset components.

Component	Size	Role
Natural user questions	500	Breadth: 425 answerable questions and 75 realistic e-commerce boundary questions that should be declined.
nopCommerce standard admin reports	20	Accuracy: the platform's own admin report list, with the platform's own report implementations as the reference.

rejection accuracy is the fraction of the 75 boundary questions correctly rejected or clarified rather than answered. For the accuracy study, a report counts as a result-set match only when the rows returned by the compiled SQL equal those of nopCommerce's own query on the same database, compared value by value over overlapping rows. Of these, only result-set match tests whether a returned answer is correct; supported execution validity tests only that the compiled SQL runs without error, which a semantically wrong query can also do, and the remaining metrics measure parser behaviour and coverage.

6.1 Evaluation Scope and Repeatability

The study uses one production-style schema, nopCommerce. It does not claim that AEGIS is an open-ended text-to-SQL system or that it can answer every possible e-commerce question. The tested claim is narrower: when the required business concepts are declared in the semantic layer and the required result shape is supported by rule-based compiler templates, AEGIS should parse the request, resolve it to approved concepts, compile safe SQL, execute the query, and produce an appropriate report. When a request depends on concepts outside that boundary, the correct behavior is to decline or ask for clarification rather than invent an answer.

All reported figures are backed by static datasets, benchmark scripts, and recorded result files. Because the evaluation includes both answerable and intentionally unsupported questions, the same dataset can be inspected and rerun rather than relying on selected examples.

6.2 Dataset and Environment

All executable evaluations run against the nopCommerce MySQL database seeded from the repository schema and mock data. The loaded database contains 1,200 customers, 2,500 orders, 6,320 order items, 1,492 shipments, 17 products, 8 categories, 8 manufacturers, and 1 store. Date-sensitive tests use the repository date-refresh script so relative phrases such as "today", "this week", and "this month" remain meaningful when the benchmark is rerun.

The evaluation dataset has two static components, summarised in Table 3:

The two components differ in who chose them. The 500 questions were written for this study, so they measure how the architecture behaves across the range of language a store owner uses. The 20 reports were not: the list is nopCommerce's own admin

Table 4 500-question live natural-language benchmark results.

Metric	Result
Parser success	499/500 (99.8%)
Supported intent exact match	313/425 (73.6%)
Supported answer rate	423/425 (99.5%)
Supported execution validity	422/425 (99.3%)
Boundary rejection accuracy	72/75 (96.0%)

menu and the comparison target is nopCommerce's own service-layer code, read from source at commit `64bdf2ff08c8b39e65717bcf974fb43dc2ef68f2` (version 5.00.0). Comparing against a platform's shipped implementation is stronger evidence than comparing against an expected-answer set written by the same authors as the system under test, because the latter tends to agree with the implementation wherever both authors reasoned the same way.

For repeatability, the released output includes the question set, the report semantics and reference queries, and the recorded output of every benchmark run under the evaluation output directory.

The 500-question dataset's 425 supported questions cover KPI, ranking, trend, segmentation, listing, time-filtered requests, item-grain substitutions, customer/order/product/geography/store/status/payment/shipping dimensions, and approved predicates such as low stock. Its 75 boundary questions remain e-commerce related but require concepts not currently modeled, such as web telemetry, marketing attribution, support tickets, review-text sentiment, forecasting, churn prediction, supplier performance, fraud scoring, delivery SLA analysis, and product affinity.

Intent extraction was served by an OpenAI-compatible gateway configured with a routing alias. The alias resolved to `gpt-5.5` for 352 of the 500 requests and to `deepseek-v4-flash` for 147; each result row records the model that actually served it. The run is therefore not pinned to a single model, and figures that depend on parser behaviour should be read with that in mind.

6.3 Evaluation A: 500-Question Live Natural-Language Benchmark

Table 4 reports the live benchmark, which sends each of the 500 static natural-language prompts through the AEGIS parser, semantic resolver, rule-based compiler, and MySQL execution path. Intent annotations are used for strict parser-slot comparison only; behavioral success is measured by whether supported questions are answered and executed and whether boundary questions are rejected or clarified.

The boundary total combines two correct behaviours that are worth separating. Of the 75 boundary questions, 60 were rejected outright and 12 were returned as clarification requests; both keep an unsupported request from being answered, but they are not the same response to a user, so collapsing them hides that the system refuses more often than it asks. The remaining three are the two carrier questions

answered by shipping method and one malformed parser reply, discussed below. The two supported questions that were refused (described below) were also correct refusals rather than failures, so of every request the pipeline declined on this run, each was one it should not have answered.

An initial pass of this benchmark lost 29 questions to a consecutive block of HTTP 502 responses from the gateway, which reduced the supported answer rate to 92.9%. Those 29 were retried and 28 answered on the second attempt. Both passes are recorded: the first is what the dataset looks like through a degraded provider, and reporting only the second would present a provider outage as though it never happened, while reporting only the first would attribute that outage to the architecture.

The exact-intent score is intentionally stricter than the behavioral measures. It requires the parser to match the annotation for class, metric, dimension, time phrase, filters, sorting, and limit. A previous run of the same dataset scored 81.2% on this measure against 73.6% here; the difference tracks the model mix described above rather than any change to the pipeline.

Two supported questions were refused. Both ask to break refunds down by payment method without naming a measure, and a segment report cannot be constructed without one. Choosing a measure on the requester's behalf is precisely the substitution the resolver is built to avoid, so these are correct refusals rather than failures.

Three boundary questions were answered rather than declined. One was a malformed model reply, which is a parse failure counted here as a boundary miss rather than a decision the system made. The other two are the same question phrased twice, asking to compare two named shipping carriers: the model reported both carrier names in `unmapped_terms` and the resolver answered by shipping method regardless. That behaviour is deliberate — a model-reported gap is treated as evidence rather than a verdict, because treating it as a verdict produced a high rate of false refusals — and these two questions are what that setting costs.

The same dataset is also executed with the model removed from the loop, by feeding each question's committed intent annotation directly to the resolver. That configuration resolves, compiles, and executes 425 of 425 supported questions and labels 75 of 75 boundary questions correctly. It is reported here only as a regression gate on the resolver and compiler; because no model participates, it is not an end-to-end result and must not be read as one.

6.4 Evaluation B: Accuracy Against nopCommerce's Own Report Logic

Each of nopCommerce's twenty standard admin reports is requested in ordinary business phrasing, with the outcome in Table 5. Two checks are applied. The first asks whether the request reaches an answer and compiles to SQL. The second executes that SQL and the platform's own query against the same seeded database and compares the returned rows.

Only the second check tests the claim. The first is satisfied by any query that compiles, and several of these twenty once passed it while being silently wrong — an order-level revenue sum fanned out across item-level joins, a missing soft-delete filter,

Table 5 Fidelity against nopCommerce's own report implementations.

Check	Result
Reached an answer and compiled to SQL	20/20
Result set matched the platform's own query	15/20 (75.0%)

Table 6 AEGIS and a direct LLM-to-SQL baseline on the same 500 questions.

Metric	AEGIS	Direct LLM-to-SQL
Supported execution validity	422/425 (99.3%)	365/425 (85.9%)
Out-of-scope questions answered	3/75 (4.0%)	25/75 (33.3%)
Queries containing a forbidden construct	0	2/500
Prompts the parser could not read	1/500	0/500

a customer breakdown grouped by display name. Each returned a plausible, chartable number, so nothing downstream could distinguish it from a correct answer.

The five reports that did not match agree with the platform's query on every value in every overlapping row. Four differ in result-set size, because the platform's own reports carry their own limits of five, fifteen or one hundred rows; two differ in the label column, returning a customer name where the platform's query labels by email address. No report returned a different value. Aligning the limits would mean adding per-report presets, which is the report-specific special-casing the semantic-layer design exists to avoid.

One portability note matters for reproduction: the reference queries reference table names in lower case while the schema creates them capitalised, so on a case-sensitive server every reference query fails outright. The row-by-row comparison was run with `lower_case_table_names=1`.

6.5 Evaluation C: Direct LLM-to-SQL Baseline

Table 6 compares AEGIS with a direct baseline: the same model, through the same gateway, was asked to write MySQL directly for the same 500 questions against the same database, with no semantic layer between the model and the SQL. This isolates the architectural variable: the arms differ only in whether the model authors the query.

The middle row carries the finding. A third of the questions the semantic layer cannot express were answered by the unconstrained model with confident, executable SQL. Asked to forecast next month's sales, it returned a query summing past months, which runs and returns a number and forecasts nothing. Asked which customers are likely to churn, it returned a query counting each customer's past orders. Asked what customers say about delivery speed, it returned raw review rows. Each answer is plausible, chartable, and addresses a different question than the one asked, which is

Table 7 Per-stage latency over the supported questions of the live benchmark.

Stage	Median	95th percentile
Intent extraction (model)	9,029.89 ms	19,460.15 ms
Semantic resolution	0.89 ms	1.25 ms
SQL compilation	0.13 ms	0.21 ms
Database execution	2.54 ms	12.87 ms
All stages after the model	3.62 ms	14.02 ms

the failure mode the architecture is designed to make structurally unreachable rather than statistically rare.

Two baseline queries contained constructs the compiler forbids, both `UNION`-based attempts to classify review text by keyword matching.

6.6 Latency

Per-stage timings were recorded for every supported question in the live benchmark and are reported in Table 7.

The split matters more than the totals. Substantially all of the wall clock is the model reading the question; the stages that resolve business concepts, decide the join path, emit SQL, and execute it account for a few milliseconds, and the model has no influence over any of them. The model figure is a property of the gateway used here and would change with the provider. The rule-based stages' figures are properties of the architecture and would follow it to another deployment. The direct baseline's generation stage has a comparable median of 9,154.02 ms, confirming that the constrained pipeline does not pay a latency premium for its safety.

6.7 Safety Evaluation

SQL safety is enforced structurally. The LLM never emits SQL. It emits a typed intent object over injected semantic-layer vocabulary; the resolver matches that object to approved metrics, dimensions, predicates, and time rules; the compiler emits parameterized SQL from rule-based templates; and the post-compilation safety monitor rejects forbidden constructs. The primary safety claim is therefore architectural rather than statistical: untrusted natural-language text is not inserted into executable SQL identifiers or clauses, and no rate of successful defence is being asserted.

The baseline comparison gives that claim an experimental counterpart. Both arms were scanned with the same forbidden-pattern set, imported from the compiler rather than restated, so neither arm is judged by a more lenient rule. The unconstrained arm produced two queries containing a forbidden construct; the constrained arm produced none, and could not have, because no path exists from an intent object to a query the templates do not generate.

6.8 Interpretation

Three conclusions follow. First, on the 500-question dataset, AEGIS answers and executes nearly all supported requests while declining the large majority of realistic out-of-boundary ones. Second, against nopCommerce's own report implementations, three quarters of the twenty reports match exactly and the remainder differ only in row count and label column, with no value discrepancy. Third, the same model without a semantic layer answers a third of the unanswerable questions anyway, which is the behaviour the architecture removes rather than reduces.

The main limitation is scope, and it is a consequence of the design rather than an implementation gap. The semantic layer is finite and deployment-specific. Adding telemetry, campaign attribution, review sentiment, forecasting, or supplier operations would require extending it and, for genuinely new result shapes, the compiler templates. This also places the work outside cross-domain benchmarks such as Spider and BIRD, which test generalisation to unseen schemas: a deployment-specific semantic layer does not generalise to an unseen schema by construction, and evaluating against those benchmarks would measure a property this architecture does not claim.

A second limitation is that correctness at scale rests on the twenty reports rather than on all 425 supported questions. For the 425, the evidence is that the compiled SQL resolves and executes, not that every returned answer is the intended one. Extending value-level verification to a stratified sample of the dataset is the most useful next measurement.

7 Discussion

The evaluation indicates that AEGIS is most useful for approved analytical reporting rather than unrestricted database exploration. There, a system must do more than produce a SQL query; the query must respect business definitions, permissions, safety constraints, and reusable reporting workflows.

7.1 Comparison with Direct LLM-to-SQL

Table 8 sets out the structural difference. Direct LLM-to-SQL systems ask the model to generate executable SQL from natural language. This can be flexible, but it leaves safety and semantic correctness dependent on model behavior. AEGIS changes the role of the model. The model extracts intent, while SQL is produced by a rule-based compiler over a semantic layer.

This difference explains the main trade-off. AEGIS gives up unlimited query flexibility in exchange for stronger control over what can be asked, how it is translated, and what is allowed to execute.

7.2 Role of the Semantic Layer

The semantic layer is not only a convenience for matching words to columns. It is the boundary of the system's analytical knowledge. If a metric, dimension, or predicate is declared, users can ask for it in many natural phrasings and combine it with other supported concepts. If it is not declared, AEGIS should not invent an answer.

Table 8 Structural comparison between direct LLM-to-SQL and AEGIS.

Property	Direct LLM-to-SQL	AEGIS
SQL generation	Model-generated	Template-compiled
Schema exposure to LLM	Usually required	Limited to approved metric and dimension names (Section 4.4)
Business definitions	Inferred from schema/context	Declared in semantic layer
Unsupported requests	May still produce SQL	Clarify or reject
Safety control	Prompting and validation	Structural constraint plus validation
Output	Query/result	Saved dashboard widget

This design is especially important for business reporting because database column names rarely capture the full business meaning of a report. For example, revenue may need to exclude deleted orders, product-level revenue may need item-grain aggregation, and customer reports may require a specific customer identity field. These definitions belong in the semantic layer rather than in ad hoc SQL generated per request.

7.3 Refusal as a Correct System Behavior

A central result of this work is that refusal must be measured, not hidden. In a bounded analytics system, some user questions are outside the implemented semantic layer even if they are reasonable business questions. The 500-question benchmark therefore includes realistic e-commerce boundary requests, and AEGIS is evaluated on whether it rejects or clarifies them.

This is different from treating every non-answer as a failure. For AEGIS, answering an unsupported question with a plausible but wrong query is worse than declining it. The explicit ANSWER / CLARIFY / REJECT outcome model is therefore part of the architecture, not only an error-handling feature.

7.4 Generality of the Architecture

The prototype is implemented and evaluated on nopCommerce with MySQL, but the architecture is not tied to that particular schema. To apply AEGIS to another system, the developer must define the semantic layer for that system and provide compiler templates for the target SQL dialect. The architecture remains the same: language understanding is separated from approved query compilation.

8 Limitations and Future Work

AEGIS is intentionally bounded. Its safety and auditability come from the fact that all answerable concepts must be declared in the semantic layer and all executable SQL must be produced by rule-based compiler templates. The limitations below should

therefore be read as explicit boundaries of the current nopCommerce implementation, not as reasons to bypass the architecture with free-form SQL generation.

- **Single-domain evaluation.** The final evaluation is over one e-commerce deployment, nopCommerce. The results show that the architecture works in this domain, but they do not prove cross-domain generality. Future work should repeat the same static-dataset process on a second schema such as WooCommerce or a non-commerce operational database.
- **Author-generated natural-language data.** The 500-question dataset is static and checkable, but it is still author-generated. A stronger study would collect questions from store owners or administrators, then annotate answerability and expected semantic bindings with at least two independent annotators.
- **Finite semantic coverage.** Boundary questions about web telemetry, marketing attribution, support tickets, review-text sentiment, forecasting, churn prediction, supplier performance, fraud scoring, delivery SLA analysis, and product affinity are deliberately outside the current semantic layer. Supporting them requires adding approved metrics, dimensions, predicates, tables, and templates.
- **Report accuracy gap.** Fifteen of nopCommerce's twenty standard admin reports match the platform's own result set exactly. The other five agree on every value but differ in row count, because the platform's reports carry their own limits, or in label column. Closing that gap by adding per-report presets would defeat the purpose of the semantic layer.
- **Correctness at scale.** Value-level verification covers the twenty reports, not all 425 supported questions; for those the evidence is that the compiled SQL resolves and executes. Extending value-level checking to a stratified sample of the dataset is the most useful next measurement.
- **Model pinning.** The gateway used for intent extraction resolved a routing alias per request, so the reported run spans two models. Each result row records the model that served it, but a single-model run is needed before parser-dependent figures can be attributed to one system.
- **LLM dependence for intent extraction.** The compiler and safety layer are rule-based, but natural-language intent extraction still depends on the configured LLM API. The live 500-question benchmark records parser success and exact intent agreement, but future work should compare multiple OpenAI-compatible models on the same static dataset.
- **Intent misinterpretation boundary.** AEGIS does not claim to perfectly infer intent from vague language. If the LLM converts an unsupported request into a plausible supported intent, rule-based validation may accept the standardized intent because it is now in the semantic-layer vocabulary. This is a prototype intent-extraction limitation, not a licence for free-form SQL: the executed query still comes only from approved semantic definitions.
- **Prototype database target.** The evaluated prototype targets nopCommerce on MySQL. This is an implementation and evaluation-scope choice, not an architectural limitation: the same semantic-layer and rule-based compilation design can support PostgreSQL, SQL Server, or other databases by adding dialect-specific compiler templates and safety rules.

- **Widget persistence.** The prototype stores widget metadata in simple local persistence. A production deployment should move the widget registry to a transactional database with migrations, ownership policies, and administrative audit views.

9 Conclusion

AEGIS is a system for turning plain-English reporting requests into dynamic, refreshable dashboard widgets over relational databases. Its contribution has three parts.

The first is architectural. The LLM is confined to understanding the question; query construction, chart selection, and widget storage are performed by fixed templates and rules downstream of it. Because the compiler emits SQL only by expanding a closed set of templates over a curated semantic layer, and never by inserting model-produced text, unsafe SQL is excluded by design rather than filtered after the fact (Section 4.7), under the assumptions stated in Section 8. The design thus shifts safety from a property the model must get right on each request to one the pipeline enforces structurally.

The second is a pair of mechanisms for the boundary of that vocabulary. Vocabulary injection removes the manually maintained synonym list and lets the model translate flexible user wording into structured semantic-layer intent. AEGIS then validates that structured intent before compilation, while retaining the original text only for narrow non-executable safety/scope cues. The ANSWER / CLARIFY / REJECT channel gives the pipeline somewhere to put the answer "this cannot be expressed here" when the structured intent is unsupported, ambiguous, destructive, sensitive, or outside the SQL-only analytical mode.

The third is evaluative. The final evaluation rests on two static nopCommerce evaluation materials rather than unsupported headline claims: a 500-question natural-language benchmark and nopCommerce's own twenty standard admin reports. On the live 500-question run, AEGIS answered 423 of 425 supported requests, executed 422, and declined 72 of 75 realistic boundary requests. Against the platform's own report implementations, fifteen of twenty result sets matched exactly and the remaining five differed only in row count and label column, with no value discrepancy. The same model without a semantic layer answered a third of the unanswerable questions anyway and produced two queries containing constructs the compiler forbids, which is the contrast the architecture is meant to produce.

AEGIS is therefore not an infinite natural-language-to-SQL engine. It is a bounded architecture for safe natural-language analytics over an approved semantic layer, suited to environments where data privacy, consistent reporting definitions, auditability, and reusable reporting widgets matter more than unlimited query flexibility.

Declarations. **Funding** No funding was received for this work.

Conflict of interest The author declares no conflict of interest.

Data availability The evaluation datasets and result artifacts are included with the released research repository.

Code availability The prototype implementation and manuscript artifacts are maintained in the accompanying repository.

Author contribution Md. Riaz designed the architecture, implemented the prototype, constructed the evaluation datasets, ran the experiments, and wrote the manuscript.

References

[1] Woods, W.A.: Progress in natural language understanding: An application to lunar geology. Proceedings of the June 4–8, 1973, National Computer Conference and Exposition, 441–450 (1973)

[2] Grosz, B.J.: TEAM: A transportable natural-language interface system. Proceedings of the First Conference on Applied Natural Language Processing, 39–45 (1983)

[3] Blunschi, L., Jossen, C., Kossmann, D., Mori, M., Stockinger, K.: SODA: Generating SQL for business users. Proceedings of the VLDB Endowment **5**(10), 932–943 (2012)

[4] Saha, D., Floratou, A., Sankaranarayanan, K., Minhas, U.F., Mittal, A.R., Özcan, F.: ATHENA: An ontology-driven system for natural language querying over relational data stores. Proceedings of the VLDB Endowment **9**(12), 1209–1220 (2016)

[5] Li, F., Jagadish, H.V.: Constructing an interactive natural language interface for relational databases. Proceedings of the VLDB Endowment **8**(1), 73–84 (2014)

[6] Affolter, K., Stockinger, K., Bernstein, A.: A comparative survey of recent natural language interfaces for databases. The VLDB Journal **28**, 793–819 (2019)

[7] Liu, M., Li, J., Wang, T., Yang, S., Liu, X.: A systematic review of natural language interfaces for databases. Frontiers of Computer Science **20**, 2011623 (2026)

[8] Zhong, V., Xiong, C., Socher, R.: Seq2SQL: Generating structured queries from natural language using reinforcement learning. In: Proceedings of the International Conference on Learning Representations (2018)

[9] Yu, T., Li, Z., Zhang, Z., Zhang, R., Radev, D.: Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, pp. 3911–3921 (2018)

[10] Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., Zhang, Z., Radev, D.: SParC: Cross-domain semantic parsing in context. In: Proceedings of the 57th Annual Meeting of the Association for

Computational Linguistics, pp. 4511–4523 (2019)

[11] Yu, T., Zhang, R., Er, H., Li, S., Xue, E., Pang, B., Lin, X.V., Tan, Y.C., Shi, T., Li, Z., Jiang, Y., Yasunaga, M., Shim, S., Chen, T., Fabbri, A.R., Li, Z., Chen, L., Zhang, Y., Dixit, S., Radev, D.: CoSQL: A conversational text-to-SQL challenge towards cross-domain natural language interfaces to databases. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing, pp. 1962–1979 (2019)

[12] Li, J., Hui, B., Qu, G., Yang, J., Li, B., Li, B., Wang, B., Qin, B., Geng, R., Huo, N., Zhou, X., Ma, C., Li, G., Chang, K.C.-C., Huang, F., Cheng, R., Li, Y.: Can large language models serve as a database interface? a big bench for large-scale database grounded text-to-SQLs. In: Advances in Neural Information Processing Systems, vol. 36 (2023)

[13] Wang, B., Shin, R., Liu, X., Polozov, O., Richardson, M.: RAT-SQL: Relation-aware schema encoding and linking for text-to-SQL parsers. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pp. 7567–7578 (2020)

[14] Scholak, T., Schucher, N., Bahdanau, D.: PICARD: Parsing incrementally for constrained auto-regressive decoding from language models. In: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, pp. 9895–9901 (2021)

[15] Pourreza, M., Rafiei, D.: DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction. In: Advances in Neural Information Processing Systems, vol. 36, pp. 28448–28472 (2023)

[16] Gao, D., *et al.*: Text-to-SQL empowered by large language models: A benchmark evaluation. Proceedings of the VLDB Endowment **17**(5), 1132–1145 (2024)

[17] Wang, B., *et al.*: MAC-SQL: A multi-agent collaborative framework for text-to-SQL. In: Proceedings of the 31st International Conference on Computational Linguistics, pp. 540–557 (2025)

[18] Talaei, S., et al.: CHESS: Contextual harnessing for efficient SQL synthesis. arXiv:2405.16755 (2024)

[19] Lei, F., *et al.*: Spider 2.0: Evaluating language models on real-world enterprise text-to-SQL workflows. In: Proceedings of the International Conference on Learning Representations (2025)

[20] Shalaan, H.S., Hammad, M., El-Attar, N.E., Elgendy, N.: G-SQL: A schema-aware and rule-guided approach for natural language to SQL. IEEE Access **13**, 158520–158534 (2025)

[21] Su, X., Zhang, Y., Wang, X., Li, Y., Liu, H.: A robust natural language text-to-SQL generation framework. Scientific Reports **16**, 7892 (2026)

[22] Zhang, Y., Dong, X., Chang, S., Yu, T., Shi, P., Zhang, R.: Did you ask a good question? a cross-domain question intention classification benchmark for text-to-SQL. (2020). arXiv:2010.12634

[23] Lee, G., *et al.*: EHRSQL: A practical text-to-SQL benchmark for electronic health records. In: Advances in Neural Information Processing Systems: Datasets and Benchmarks Track, vol. 35 (2022)

[24] Narechania, A., Srinivasan, A., Stasko, J.: nl4dv: A toolkit for generating analytic specifications for data visualization from natural language queries. IEEE Transactions on Visualization and Computer Graphics **27**(2), 369–379 (2021)

[25] Luo, Y., Tang, N., Li, G., Tang, J., Chai, C., Qin, X.: Synthesizing natural language to visualization benchmarks from NL2SQL benchmarks. In: Proceedings of the ACM SIGMOD International Conference on Management of Data, pp. 1235–1247 (2021)

[26] Setlur, V., Battersby, S.E., Tory, M., Gossweiler, R., Chang, A.X.: Eviza: A natural language interface for visual analysis. In: Proceedings of the 29th Annual ACM Symposium on User Interface Software and Technology, pp. 365–377 (2016)

[27] Gao, T., Dontcheva, M., Adar, E., Liu, Z., Karahalios, K.G.: Datatone: Managing ambiguity in natural language interfaces for data visualization. In: Proceedings of the 28th Annual ACM Symposium on User Interface Software and Technology, pp. 489–500 (2015)

[28] Deng, D., Wu, A., Qu, H., Wu, Y.: Dashbot: Insight-driven dashboard generation based on deep reinforcement learning. IEEE Transactions on Visualization and Computer Graphics **29**(1), 690–700 (2023)

[29] Wu, A., Wang, Y., Zhou, M., He, X., Qu, H.: Multivision: Designing analytical dashboards with deep learning based recommendation. IEEE Transactions on Visualization and Computer Graphics **28**(1), 162–172 (2022)

[30] Wang, Y., Sun, Z., Zhang, H., Cui, W., Xu, K., Ma, X., Zhang, D.: Datashot: Automatic generation of fact sheets from tabular data. IEEE Transactions on Visualization and Computer Graphics **26**(1), 895–905 (2020)

[31] Shi, D., Xu, X., Sun, F., Shi, Y., Cao, N.: Calliope: Automatic visual data story generation from a spreadsheet. IEEE Transactions on Visualization and Computer Graphics **27**(2), 464–474 (2021)

[32] Lehmann, C., Gehrig, D., Holdener, S., Saladin, C., Monteiro, J.P., Stockinger,

K.: Building natural language interfaces for databases in practice. In: Proceedings of the 34th International Conference on Scientific and Statistical Database Management (2022)

[33] Pedro, R., Castro, D., Carreira, P., Santos, N.: Prompt-to-SQL injections. In: Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (2025)

[34] OpenAI: Introducing structured outputs in the API (2024)